

Zero to Hero Study Handbook: Medical Document Intelligence Assistant

Static-analysis note: this handbook is based only on repository source/config/docs inspection in ``/home/ahmad/AI/medical-document-intelligence-assistant``.

Module 1: Foundations & Architecture

- This project is a local-first medical document intelligence platform for educational document understanding.
- It supports upload, OCR/parsing, medical entity extraction, timeline construction, hybrid retrieval, grounded QA, summaries, and doctor-visit report generation.
- It explicitly enforces educational-only safety behavior and blocks diagnosis/treatment/prescription advice.

High-level use cases

- Upload prescriptions/lab reports/scanned documents/handwritten notes and structure them.
- Ask grounded questions with citations back to document chunks/pages.
- Generate timeline views and report artifacts for clinician discussion prep.
- Manage model routing/memory/history in a local deployment.

Core paradigms and patterns (as implemented here)

- Layered modular backend: routers -> services -> models/utilities.
- Graph-based orchestration:
- Medical workflow via ``MedicalSupervisor`` LangGraph in ``services/medical/agents.py``.
- Extraction workflow via ``extraction_graph`` LangGraph in ``services/extraction/graph.py``.
- Hybrid retrieval: weighted semantic cosine + keyword overlap in ``services/medical/retrieval.py``.
- Rule-based model routing: ``ModelRouter.route(task)`` in ``services/infrastructure/model_router.py``.
- Local-first privacy controls:
- Upload encryption-at-rest in ``routers/documents.py`` + ``security/crypto.py``.
- Local Ollama URL guard in ``utils/network.py``.
- Safety-guarded generation:
- Prohibited request detection and disclaimer injection in ``services/medical/safety.py``.

Architecture description (main runtime path)

- Frontend (Next.js App Router) calls ``/api/*`` via rewrite proxy.
- Backend (FastAPI) exposes medical, documents, auth, extraction, and provider APIs.
- Processing pipeline:
- Upload -> encrypted storage
- OCR -> entity/lab/med extraction -> chunking -> embeddings
- persistence to relational tables (including chunk embeddings metadata)
- retrieval/QA/summary/timeline/report APIs over persisted data.
- Storage:

- Relational DB via SQLAlchemy/Alembic (SQLite/Postgres supported).
- “Vector store” behavior is implemented through `document\_chunks.embedding` plus app-level retrieval scoring.

Main flow ASCII diagram

```

Browser (Next.js)
|
| /api/documents (multipart upload)
v
FastAPI Router: documents.upload_document
-> save_upload + magic-byte validation + AES-GCM encrypt
-> documents table row
|
| /api/medical/process/{document_id}
v
MedicalPipelineService.process_document
-> MedicalSupervisor.execute (LangGraph)
    1) ocr_agent
    2) entity_agent
    3) timeline_agent
    4) retrieval_agent
    5) parallel_clinical_agents
    6) memory_agent
-> persist_pipeline_outputs
    -> ocr_pages, medical_entities, lab_results, medication_history,
        timeline_events, document_chunks(+embedding)
|
+--> /api/search
+--> /api/qa/query or /api/qa/query/stream
+--> /api/summaries
+--> /api/timelines
+--> /api/reports/generate

```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Python package metadata, deps, scripts	project.scripts` (`mdia-api`, `mdia-mcp`)	
justfile	Canonical dev/build/test/run command aliases	`install`, `sync`, `dev`, `serve`, `test`, `mi`	
apps/backend/app/main.py	FastAPI app assembly, lifespan, health/metrics/info	`lifespan`, `_recover_o`	
apps/backend/app/config.py	Central typed settings	`Settings` DB, Ollama, auth, routing, disclaimer	
apps/backend/app/database.py	Async DB engine/session, migration bootstrap	`get_db`, `init_db` `DAT`	
apps/backend/app/routers/documents.py	Upload/list/get/delete document APIs	`upload_document` magic	
apps/backend/app/routers/medical.py	Main medical assistant API surface	`process_document`, `hybrid_s`	
apps/backend/app/routers/auth.py	Authentication endpoints	`bootstrap_admin`, `login`, `refresh_token`	
apps/backend/app/services/medical/pipeline.py	End-to-end medical processing orchestration	`MedicalPi`	
apps/backend/app/services/medical/agents.py	LangGraph supervisor + specialized agents	`MedicalSuperv`	
apps/backend/app/services/medical/extraction.py	Regex/rule-based medical extraction	`extract_entitie`	
apps/backend/app/services/medical/chunking.py	Section-aware chunk generation	`chunk_pages` date me	
apps/backend/app/services/medical/retrieval.py	Hybrid retrieval and chunk indexing	`HybridRetriever`	
apps/backend/app/services/medical/qa.py	Grounded QA generation and session persistence	`MedicalQASer`	
apps/backend/app/services/medical/summarization.py	Summary generation	`SummaryService.summarize` s	
apps/backend/app/services/medical/reporting.py	Doctor report generation/export	`ReportService.genera`	
apps/backend/app/services/medical/safety.py	Educational-only policy logic	`is_prohibited_medical_req`	
apps/backend/app/services/infrastructure/model_router.py	Task-to-model routing	`ModelRouter.route`	
apps/backend/app/services/infrastructure/ollama_client.py	Ollama HTTP wrapper	`generate`, `generate`	
apps/backend/app/services/ocr/registry.py	OCR provider registry/auto routing	`get_ocr_provider`, `li`	
apps/backend/app/models/medical_db_models.py	Medical assistant persistence models	`DocumentChunk`, `	
apps/backend/app/models/medical_schemas.py	Pydantic API contracts	`ProcessDocumentResponse`, `Search`	
apps/frontend/src/lib/api.ts	Typed frontend API client and SSE parser	`uploadDocument`, `streamQuest`	
apps/frontend/src/components/AppFrame.tsx	Global app shell/navigation	`NAV_ITEMS`, `AppFrame` page	
apps/frontend/src/app/upload-center/page.tsx	Upload UX + process trigger	`submit` sequential uploa	
apps/frontend/src/app/ai-chat/page.tsx	Streaming chat UI	`submit` with SSE handlers session contin	
apps/frontend/src/components/EvidenceViewer.tsx	Citation-to-page synchronized viewer	`computeHighlig`	
Dockerfile	Backend container build	uv builder/runtime stages `UPLOAD_DIR`, `ARTIFACTS_DIR`	
docker-compose.yml	Multi-service local stack	backend/frontend/ollama/postgres/redis/monitoring env	

Module 3: Core Execution Flows

Flow 1: Startup and readiness

1. Backend entrypoint is FastAPI app in `app.main:app`.
2. `lifespan()` validates local Ollama URL, initializes DB, loads model config, sets telemetry, recovers orphaned jobs.
3. Routers are mounted: auth, documents, schemas, extractions, providers, medical.
4. Health/metrics/info endpoints expose readiness and runtime state.

Flow 2: Main medical document pipeline

1. Frontend upload page calls `POST /api/documents/`, then `POST /api/medical/process/{document\_id}`.
2. `upload\_document()` validates upload + MIME magic bytes, encrypts file, writes `Document` row.
3. `MedicalPipelineService.process\_document()` creates agent run, decrypts temp OCR input when needed, invokes `MedicalSupervisor` graph.
4. `MedicalSupervisor` runs OCR -> entity extraction -> timeline -> retrieval chunk prep -> parallel clinical preview agents -> memory stage.
5. `persist\_pipeline\_outputs()` stores OCR pages/entities/labs/medications/timeline/chunks and embeddings metadata.

Flow 3: Search + grounded QA

1. `POST /api/search` executes hybrid retrieval with semantic+keyword scores and optional filters.
2. `POST /api/qa/query`:
 - blocks prohibited medical requests.
 - builds grounded context from retrieved chunks.
 - generates answer with Ollama model routing.
 - appends disclaimer and returns citations.
3. `POST /api/qa/query/stream` emits SSE events: `session`, repeated `token`, final `done`.

Flow 4: Summaries, timelines, and reports

1. `POST /api/summaries` uses chunk context + model routing and returns citations + safety.
2. `POST /api/timelines` filters `timeline\_events` by docs/event type/date range.
3. `POST /api/reports/generate` composes markdown/html/json payload and persists `GeneratedReport`.
4. `GET /api/reports/{id}/export` supports `markdown`, `html`, `json`, and `pdf` (base64 content).

Flow 5: Auth, memory, model manager, health

- Auth: `/api/auth/bootstrap`, `/api/auth/login`, `/api/auth/refresh`.
- Memory: `POST/GET/DELETE /api/memory`.
- Model config: `GET/PATCH /api/models/config` (admin-gated patch).
- Health monitor: `GET /api/system/health` with GPU/Ollama/process memory stats.
- Disclaimer API: `GET /api/medical/disclaimer`.

Key input/output shapes

Search request (`SearchRequest`):

```
{
  "query": "string",
  "top_k": 10,
```

```

    "document_ids": ["doc_id"],
    "start_date": "2026-06-01",
    "end_date": "2026-06-30",
    "filters": {"min_score": 0.1}
}

```

QA response (`QAResponse`):

```

{
  "session_id": "string",
  "answer": "...",
  "extracted_information": "...",
  "educational_background": "...",
  "citations": [{"document_id": "...", "page_number": 1, "evidence_text": "..."}],
  "safety": {"disclaimer": "...", "educational_use_only": true, "prohibited_actions": []},
  "model": "qwen3.5:4b"
}

```

Module 4: Setup & Run Guide

Prerequisites

- Python 3.12.x (required by `pyproject.toml`)
- `uv`
- Node.js + npm
- Ollama local service

Install commands

```

cd /home/ahmad/AI/medical-document-intelligence-assistant
uv venv --python 3.12.10 .venv
source .venv/bin/activate
uv sync --frozen --extra test --extra lint
cd apps/frontend
npm install
cd ../../

```

Environment config

Create backend env file:

```
cp apps/backend/.env.example apps/backend/.env
```

Important key groups:

- Database: `DATABASE\_URL`, `SYNC\_DATABASE\_URL`
- Storage/security: `UPLOAD\_DIR`, `ARTIFACTS\_DIR`, `STORAGE\_ENCRYPTION\_KEY`
- OCR/Ollama: `ENABLE\_GLM\_OCR`, `ENABLE\_PADDLEOCR`, `OLLAMA\_BASE\_URL`, `OLLAMA\_GLM\_OCR\_MODEL`
- Routing models: `DEFAULT\_CHAT\_MODEL`, `SUMMARY\_MODEL`, `ENTITY\_MODEL`, `EMBEDDING\_MODEL`, `FALLBACK\_CHAT\_MODELS`
- Auth/security: `ENABLE\_AUTH`, `JWT\_SECRET\_KEY`, token expiry values
- Policy: `OFFLINE\_BY\_DEFAULT`, `ALLOW\_EXTERNAL\_NETWORK`, `MEMORY\_RETENTION\_DAYS`

Migration and startup

```
alembic upgrade head
```

Run backend:

```
uv run uvicorn app.main:app --app-dir apps/backend --host 0.0.0.0 --port 8000
```

Run frontend:

```
cd apps/frontend
npm run dev
```

Docker path

```
docker compose up --build
```

Exporting handbook markdown to PDF

Preferred if available:

```
pandoc docs/ZERO_TO_HERO_STUDY_HANDBOOK.md -o docs/ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

Ordered study plan

1. Start with ``pyproject.toml``, ``justfile``, ``apps/backend/app/config.py``, ``apps/backend/app/main.py``.
2. Read API contracts in ``apps/backend/app/models/medical_schemas.py`` and routers under ``apps/backend/app/routers/``.
3. Read persistence models in ``apps/backend/app/models/db_models.py`` and ``medical_db_models.py``.
4. Read pipeline and graph internals in ``services/medical/pipeline.py`` and ``services/medical/agents.py``.
5. Read extraction and retrieval core in ``services/medical/extraction.py``, ``chunking.py``, ``retrieval.py``, ``qa.py``, ``reporting.py``, ``safety.py``.
6. Finish with frontend wiring in ``apps/frontend/src/lib/api.ts`` and primary pages.

Exercises

1. Trace upload security end-to-end.
2. Explain prohibited request behavior in both ``/qa/query`` and ``/qa/query/stream``.
3. Locate and explain lab out-of-range logic.
4. Explain where embeddings are generated and stored.
5. Explain ``ModelRouter`` task routing behavior.
6. Map one frontend page action to exact backend endpoint and schema.
7. Explain how report export supports 4 formats.
8. Compare medical supervisor graph vs extraction graph.

Solution outlines

1. Upload path: ``upload_document`` -> ``save_upload`` -> ``sniff_mime`` / `mime_matches_extension`` -> ``EncryptionService.encrypt_bytes`` -> ``Document`` row.
2. Guardrails: ``is_prohibited_medical_request`` gates generation and returns blocked educational response + disclaimer.
3. Out-of-range: ``_is_out_of_range(value_text, reference_range)`` in ``services/medical/extraction.py``.
4. Embeddings: ``HybridRetriever.embed_text`` + ``index_chunks`` writing ``DocumentChunk.embedding``.
5. Router: ``ModelRouter._candidate_chain(task)`` chooses preferred + fallback models, with availability/headroom checks.
6. Example: Upload Center calls ``/api/documents/`` then ``/api/medical/process/{id}`` via ``api.ts``.

7. Reports: `ReportService.export_report_content` returns markdown/html/json or base64 PDF.
8. Graphs: medical graph is domain workflow; extraction graph is schema-driven validation/review workflow.

Learner Verification Checklist

- Can you explain backend startup (`lifespan`) and readiness checks?
- Can you trace upload validation + encryption in code?
- Can you explain each `MedicalSupervisor` node and persisted outputs?
- Can you explain hybrid retrieval scoring and filter behavior?
- Can you explain grounded QA citations and disclaimer behavior?
- Can you map at least three frontend pages to backend endpoints?
- Can you explain memory, model config, and health endpoints?
- Can you explain how Alembic migration `0005_medical_assistant_platform` aligns with ORM models?